

CSCE 748 COMPUTATIONAL PHOTOGRAPHY · MODULE 06 · LECTURE NOTES

Deconvolution and Deblurring

Undoing a blur, and why it is mostly impossible.

MODULE OUTCOMES

Model blur as convolution with a point spread function.

Explain why naive inverse filtering fails.

Implement Wiener and Richardson–Lucy deconvolution.

Explain blind deconvolution and why it is ill-posed.

Explain coded apertures and flutter shutters.

1 The forward model

$$\mathbf{b} = \mathbf{k} * \mathbf{s} + \mathbf{n} \qquad \mathbf{B} = \mathbf{K} \cdot \mathbf{S} + \mathbf{N}$$

The blurred observation $\mathbf{b}$ is the sharp image $\mathbf{s}$ convolved with a **point spread function** $\mathbf{k}$, plus noise. The PSF is the image a single point of light would produce — a disc for defocus, a curve for camera shake, a line for linear motion. In the frequency domain the convolution becomes a multiplication, which is why deconvolution is attempted there.

NAIVE INVERSE FILTERING AMPLIFIES NOISE WITHOUT BOUND

The obvious inverse is $\mathbf{S} = \mathbf{B}/\mathbf{K}$. It fails comprehensively, for a reason worth understanding precisely.

A blur is a low-pass filter, so $\mathbf{K}$ is small at high frequencies and may be exactly zero at some. That is what blurring means.

Dividing by a near-zero number amplifies whatever is in the numerator — and at those frequencies the numerator is almost entirely noise, because the signal was attenuated and the noise was not.

So the 'recovered' image is dominated by enormously amplified noise, typically unrecognisable and far worse than the blurred input. This is not a numerical issue to be worked around; it is the structure of the problem.

And where $\mathbf{K}$ is exactly zero, the information is genuinely gone. Those frequencies contributed nothing to the measurement, so nothing in the measurement constrains them. **No algorithm recovers them** — anything that appears to is inventing, which is Module 12's subject.

2 Regularised deconvolution

$$\mathbf{S} = \mathbf{B} \cdot \mathbf{K}^* / (|\mathbf{K}|^2 + 1/\text{SNR})$$

Wiener deconvolution is the optimal linear filter under a Gaussian noise assumption. The added term in the denominator prevents the division from blowing up where $\mathbf{K}$ is small. Its behaviour is best understood frequency by frequency:

Regime	Behaviour	Interpretation
High SNR ($\mathbf{K}$ large, noise small)	The $1/\text{SNR}$ term is negligible, so the filter approaches $1/\mathbf{K}$.	Invert fully — the measurement is reliable here.
Low SNR ($\mathbf{K}$ small, noise dominant)	The $ \mathbf{K} ^2$ term is negligible, so the filter approaches $\mathbf{K}^* \cdot \text{SNR}$, which is small.	Suppress rather than amplify — the measurement says nothing reliable here, so do not trust it.

The single parameter is the noise level, which can be measured (Module 01) rather than guessed — which is unusually comfortable for a regularisation parameter.

RICHARDSON-LUCY

An iterative maximum-likelihood method assuming Poisson noise — which, per Module 01, is the physically correct model for photon counting, where Wiener's Gaussian assumption is an approximation.

Each iteration blurs the current estimate, compares it against the observation, and applies a multiplicative correction. The multiplicative form is what keeps it non-negative.

Non-negativity is preserved automatically, which is physically correct — radiance cannot be negative — and which Wiener deconvolution does not guarantee. Negative values in a Wiener result are a visible artefact around strong edges.

And it amplifies noise as the iteration count rises. It converges to the maximum-likelihood solution, which for noisy data is itself noisy — so the iteration must be stopped early. The stopping point is a tuning parameter disguised as a convergence criterion, and being clear about that is more honest than treating it as a property of the algorithm.

Prior	Assumption	Effect on the result
Tikhonov (L2 on gradients)	The image is smooth.	Blurs edges — which is the wrong prior for photographs, whose defining feature is that they have edges.
Total variation (L1 on gradients)	The image is piecewise constant.	Preserves edges well and flattens texture into cartoon-like regions, which is the characteristic TV artefact.
Sparse gradients (hyper-Laplacian)	Natural images have mostly small gradients and a few large ones — a heavy-tailed distribution, which is an empirically measured property rather than an assumption of convenience.	Matches real image statistics closely , and this is the prior that made blind deblurring work.

Prior	Assumption	Effect on the result
Learned	Whatever the training distribution contained.	Best measured results, and it synthesises detail consistent with the training data rather than with this image (Module 12).

3 Blind deconvolution

BLIND DECONVOLUTION IS SEVERELY ILL-POSED

You observe only b , and must recover both the kernel k and the sharp image s . The unknowns vastly outnumber the measurements.

And there is a trivial, wrong, perfectly consistent solution: k = a delta function and $s = b$. The 'sharp' image is the blurred one and the 'kernel' does nothing, and this explains the observation exactly.

So the problem is driven entirely by the priors. Without a reason to prefer a genuinely sharp image and a plausible blur kernel, the trivial solution is as good as any other — and in fact it maximises the likelihood, because it fits the data perfectly.

Fergus et al.'s 2006 paper was the breakthrough: use the heavy-tailed gradient statistics of natural images as a prior. A truly sharp image has a few very large gradients and many near-zero ones; a blurred image has many medium ones. **Under that prior the delta solution becomes highly improbable**, and the optimisation moves toward a real answer. This single observation turned blind deblurring from impossible into merely difficult.

- **Natural image priors** — sparse, heavy-tailed gradients. The core idea above.
- **Kernel priors.** A camera-shake kernel is sparse, connected, non-negative, and sums to one — it is the trace of a continuous path, not an arbitrary cloud of values. Constraining to that family removes a great deal of the search space.
- **Coarse-to-fine estimation** (Module 02): estimate a small kernel at low resolution where the problem is better conditioned, then refine at each finer scale. The same strategy as alignment and flow.
- **Edge prediction.** Predict where strong edges ought to be — by shock filtering, or from a learned estimate — and use the discrepancy between predicted and observed edges to constrain the kernel.
- **Or measure the blur directly.** An inertial sensor records the camera's motion during the exposure, which gives the kernel without any inference at all. **Measurement beats inference whenever it is available**, and this is the practical answer in any device that has an IMU — which is every phone.

4 Coded imaging

	Coded aperture	Flutter shutter
Modification	A patterned mask placed in the lens aperture.	The shutter is opened and closed in a coded binary sequence during the exposure.
Effect on the PSF	The defocus PSF becomes broadband — its spectrum has no zeros, where a plain disc aperture's spectrum has many.	The motion blur PSF becomes broadband — a plain box kernel (continuous exposure) has regularly spaced spectral zeros; a coded one does not.

	Coded aperture	Flutter shutter
Consequence	Defocus deconvolution becomes well-conditioned , and the scale of the PSF reveals depth — so a single image yields both a deblurred result and a depth map.	Motion blur becomes invertible , so a long exposure of a moving subject can be deblurred.
Cost	The mask blocks light — typically around half.	Half the light, and the motion must still be known or estimated.

DESIGN THE MEASUREMENT, NOT JUST THE ALGORITHM

A conventional camera is designed to form a good image directly. When that image is degraded, you are left with an ill-posed inverse problem and whatever priors you can muster.

A coded camera is designed so that the inverse problem is easy — deliberately capturing a *worse* raw image in exchange for one from which more can be recovered. The coded aperture image looks bad; the deconvolved result is better than anything the plain aperture could have given.

This is the central move of computational photography as a field: co-design the optics and the computation, rather than treating the camera as a fixed device whose output must be repaired afterwards.

Light field cameras (Module 09) are the same move — capture angular information instead of a focused image, and compute focus afterwards. **So is burst photography (Module 10)** — capture fifteen bad frames instead of one good one. **In each case what the sensor records is not a photograph**, and the photograph is computed from it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-463 — deconvolution and coded photography lectures (free)	Wiener, Richardson–Lucy, and the coded aperture material, with the assignment.
Fergus et al. — Removing Camera Shake from a Single Photograph (free)	The §3 breakthrough. The natural image prior argument is the part to read closely.
Levin et al. — Image and Depth from a Conventional Camera with a Coded Aperture (free)	The coded aperture of §4, including the depth recovery.
Raskar, Agrawal & Tumblin — Coded Exposure Photography: Motion Deblurring using Fluttered Shutter (free)	The flutter shutter. The spectral argument is clearly made.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Blur an image with a known kernel, add noise, and attempt naive inverse filtering. Report how bad the result is and explain it from the spectrum.
- 2 Plot the frequency spectrum of a disc PSF and a box motion PSF and identify the zeros.
- 3 Implement Wiener deconvolution and sweep the noise parameter. Find the value that works and note that it matches your measured noise.
- 4 Implement Richardson–Lucy and plot image quality against iteration count. Identify where it starts getting worse.
- 5 Implement total variation regularised deconvolution and compare against Wiener on an image with strong edges.
- 6 Measure the gradient distribution of a natural photograph and confirm it is heavy-tailed. This is the empirical fact behind §2's prior.
- 7 Attempt blind deconvolution with no prior and demonstrate that it converges to the delta-function solution.
- 8 Simulate a coded aperture: convolve with a coded PSF and with a disc of the same area, then deconvolve both. Compare.
- 9 Simulate a flutter shutter on a moving object and deconvolve. Compare against a conventional exposure.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the blur model and say what the PSF is.
- 2 Why does naive inverse filtering fail, and what happens where the spectrum is exactly zero?
- 3 Write the Wiener filter and describe its behaviour at high and low SNR.
- 4 What noise model does Richardson–Lucy assume, what does it preserve, and why must it be stopped early?
- 5 Name four priors and say which matches natural images and why.
- 6 Why is blind deconvolution ill-posed, and what is the trivial wrong solution?
- 7 What made blind deblurring work, and what is the practical alternative to inference?
- 8 What do coded apertures and flutter shutters have in common?
- 9 State the central move of computational photography as a field.